Combining the above two insights, our main result stated informally is that, given a fixed (2-layer) Transformer architecture, (a) if the underlying task requires N parallel operations, where N exceeds the number of input tokens L, and (b) as long as N is not much larger than the parameter count K of the attention-feedforward block, pause-inference can solve tasks that standard inference cannot.

We formalize the above insights in the form of assumptions stated in an abstract setting (in order to be as general as possible). We emphasize that the crux of our argument lies within these assumptions themselves, rather than the proof of our theorem. Thus, our main result here should be viewed as identifying precisely what assumptions are required for pause-inference to help.

J.1 Underlying task

We consider an abstract set of tasks that require a first step that involve multiple parallel operations, following by a simple aggregation step to arrive at the solution:

Assumption J.1. ***(structure of the underlying task)*** *Given the vocabulary space $\mathcal{V}$, let $\circ$ be a generic 2-ary operator on $\mathcal{V}$. For an input sequence length L, consider a corresponding function class $\mathcal{F}_N$ that corresponds to all functions $f : \mathcal{V}^L \mapsto \mathcal{V}$ that require applying N $\circ$ operations independently following by a generic aggregation operation $g_{\text{aggr}} : \mathcal{V}^N \to \mathcal{V}$:*

$$\mathcal{F}_N = \Bigg\{ f : \mathcal{V}^L \mapsto \mathcal{V} \Bigg| \exists i_1, \dots i_N, j_1, \dots j_N \in [1, L], \tag{18}$$

$$f(v_1, v_2, \dots, v_L) = g_{aggr}\big(\underbrace{(v_{i_1} \circ v_{j_1}), (v_{i_2} \circ v_{j_2}), \dots, (v_{i_N} \circ v_{j_N})}_{N \textit{ independent } \circ \textit{ operations.}} \big) \Bigg\}. \tag{19}$$

Examples. This structure covers a broad range of examples.

- As a simple mathematical example, this covers learning polynomials of the form $(x_1+x_2)\cdot(x_3+x_4)\cdot(x_1+x_3)$.
- As a natural language example, consider a multi-choice question-answer task with C choices given along with E pieces of evidence in the context. One can then imagine that each v_i corresponds to a piece of evidence, and each v_j a choice. We then require $N = C{\cdot}E$ operations that compare each of the given choices against each of the given pieces of evidence. A final aggregation step would select the choice for which there exists a piece of evidence the most confidently corroborates the choice.

J.2 (Tight) Upper bounds on the Transformer capacity

If the attention-feedforward module had, say, an infinite or exceedingly large capacity, the Transformer would be able to trivially express the solution to any task. It is only when these modules have finite capacity — as they do in practice — that we expect additional operations introduced by the pause tokens to be helpful. Correspondingly, we state this intuitive assumption as multiple "tight upper bound" assumptions. Each assumption states that the modules in a Transformer *can* represent objects of a certain complexity, but none any more complex than that.

Our first such assumption is in how much information can be represented by each intermediate vector. Specifically, we assume that each vector can precisely capture one token in $\mathcal{V}$ along with the positional index of the token (akin to positional embeddings injected into each token in practice). The precise form by which this information is represented as a vector is abstracted away for our discussion (e.g., it may be in one-hot form). Also note that our argument can be extended to settings where each intermediate vector could potentially represent more tokens, we discuss this at the end of the section.

Assumption J.2. ***(information bandwidth limit of intermediate Transformer operations)*** *We assume that the i'th intermediate vector in any given layer can be represented as $(u_i, i) \in \mathcal{V} \times \mathbb{N}$.*

Next, we assume a finite limit on the class of functions that each intermediate Transformer operation can represent. To state this, let $\mathbf{u} = ((u_1, 1), (u_2, 2), \dots, (u_L, L))$ be the outputs of an intermediate

layer (corresponding to a L-length input sequence). For convenience, ignoring the residual and layer-norm blocks, let the i'th output of the next layer be expressed as:

$$\phi_{\text{FF}}(\phi_{\text{Attn}}(\mathbf{u}, \mathbf{u}, (u_i, i))) = (u'_i, i) \tag{20}$$

where the first two arguments represent the keys and values, and the third argument the query for the i'th intermediate operation in the considered layer. Note that ϕ_{FF} and ϕ_{Attn} are parameterized modules that can implement a finite set of functions. We assume what this set of functions consists of:

Assumption J.3. ***(representational limits of intermediate Transformer operations)*** *We assume that for each index $i \in \mathbb{N}$, $\phi_{\text{FF}}(\phi_{\text{Attn}}(\cdot, \cdot, (\cdot, i))$ can represent exactly one of two types of functions:*

- *A **single** $\circ$ operation. Specifically, we assume the self-attention operation can **select two indices** as $\phi_{\text{Attn}}(\mathbf{u}, \mathbf{u}, (u_i, i)) = (u_{\nu(i)}, u_{\nu'(i)})$, where $\nu, \nu' : \mathbb{N} \to \mathbb{N}$ come from some finite set of "index-selecting" functions $\mathcal{P}$. We then assume ϕ_{FF} can **implement** $\circ$ as $\phi_{\text{FF}}(u_{\nu(i)}, u_{\nu'(i)}) = u_{\nu(i)} \circ u_{\nu'(i)}$.*
- *The aggregating function g_{agg} as $\phi_{\text{FF}}(\phi_{\text{Attn}}(\mathbf{u}, \mathbf{u}, (u_i, i))) = (g_{\text{aggr}}(u_1, \ldots, u_L), i)$.*

We explain why the above sub-assumptions are both reasonable and can hold simultaneously. First, we argue why it is reasonable that each intermediate Transformer operation can implement a limited number of $\circ$ operations, but not any more. Assume that the model needs to represent $u'_{35} = u_1 \circ u_3$. This requires the model to pay attention to the query's positional index 35, and then select the values at two different positional indices 1 and 3. Selecting these two values can be implemented by 2 self-attention heads operating independently. A subsequent feedforward network can then operate on a concatenated input (u_1, u_3). Crucially, implementing any further $\circ$ operations, would require more attention heads. Thus, it is reasonable to assume a (tight) limit on the number of $\circ$ attention operations.

Note that the above assumption can simultaneously hold with the assumption that the Transformer operation can implement the aggregation function g_{aggr}. This is because g_{aggr} does not require preferentially selecting any positional indices: all inputs are aggregated equally. Thus, we only need a single self-attention head that applies equal weight to all the values.

The last and arguably most insightful assumption stipulates a tight upper limit on what the self-attention module can represent. Specifically, observe that above we assume that the self-attention module can implement a finite set of functions $\mathcal{P}$, which help select "indices" $\nu(i)$ and $\nu'(i)$. We assume that there is a limit to this set of index-selecting functions $\mathcal{P}$. *Our key insight is that this limit is purely determined by the parameter count of the module, and is therefore independent of the input-sequence-length, L.* In practice, the size of the set $\mathcal{P}$ is much larger than the number of tokens L, and thus corresponds to the untapped capacity in the model. This capacity is however bottlenecked by the input length L, which determines how many operations in $\mathcal{P}$ are executed in standard inference.

Assumption J.4. ***(the "untapped" capacity of self-attention operation is independent of input length)*** *We assume that for some $K \gg L$, the self-attention module in each layer has at least $2K \log K$ parameters, and can hence implement the index-selecting functions $\nu, \nu' : [K] \to [K]$ to be any of the K^K many mappings possible. In other words, $\mathcal{P} = \{1, 2, \ldots, K\}^{\{1,2,\ldots,K\}}$.*

From our assumptions, we derive the following result on what pause-inference can implement, which standard inference cannot, given a 2-layer Transformer (we discuss extensions to larger architectures in a subsequent remark):

Theorem J.1. *Under Assumptions J.1, J.2, J.3 and J.4, standard inference on a 2-layer Transformer with no pause tokens can only represent the function class $\mathcal{F}_N$ for $N \leq L$, where N denotes the number of parallel operations required by the function class, and L denotes the length of the input sequence. In contrast, a 2-layer Transformer with $N - L$ appended dummy tokens can represent the function class $\mathcal{F}_N$ for any large $N \geq L$ as long as $N \leq K$, where K scales with the parameter count of the self-attention module as in Assumption J.4.*

The key insight is that the self-attention module has the representational capacity to implement K different $\circ$ operations, a capacity that is independent of, and much larger than the number of

input tokens. However the standard Transformer that sees only L tokens, only allows the model to realize L of these. We hope this serves as a preliminary way of formalization the notion of the "implementation capacity" of a Transformer vs. the notion of raw parameter capacity.

Proof. Under Assumption J.2 that each intermediate vector can only represent one token from $\mathcal{V}$, the first layer in the Transformer will have to implement all the N $\circ$ operations to represent any given $f \in \mathcal{F}_L$. Therefore, at the k'th index, the model would have to instantiate $\nu(i) = i_k$ and $\nu'(i) = j_k$ (as defined in Assumption J.3), so that the corresponding Transformer operation can compute $v_{i_k} \circ v_{j_k}$. However, by Assumption J.3 (which states that each intermediate Transformer operation can implement only one $\circ$ operation) and by how standard inference in a Transformer is defined (which allows only as many operations as there are input tokens), the first layer can only compute at most L many $\circ$ operations. Hence, a Transformer with standard inference can only represent $\mathcal{F}_N$ for $N \leq L$. On the other hand, from Assumption J.4, we know that the self-attention operator can implement as many as K such operations, where $K \gg L$. Thus as long as $N \leq K$, with $N - L$ dummy tokens, the Transformer can implement $\mathcal{F}_N$ for $N > L$. □

Remark J.2. ***(breaking the information bandwidth assumption)*** *In Assumption J.2, recall that we assume that in each intermediate vector, we are able to communicate precisely one token from $\mathcal{V}$. If this assumption were to break, then standard inference would be able to implement a larger class of functions where $N > L$. However, it would still fall short of what pause inference can do. Specifically, imagine that each layer passes on its computed output, and also all computed outputs from the previous layer. Thus, if the intermediate vector can represent upto N/L tokens, then the Transformer could divide the N required $\circ$ operations over N/L many layers, each layer performing L operations in parallel. This would however be slower as it requires a series of N/L computations. In contrast, pause-inference requires only 1 set of parallel computations, and also a meagre information bandwidth of 1 token per intermediate vector.*

Remark J.3. ***(recurrent depth)*** *An alternative way to exploit the untapped capacity of the attention-feedforward modules, is to repeat these operations sequentially along the depth of the model as done in Dehghani et al. (2019). This strategy would be helpful in tasks that require recursion. However, our class of problems do not involve recursion of any form. Thus, if these recurrent layers simply repeat the same $\circ$ operations over and over, we may not enjoy any advantages. However, one may argue that, perhaps repeating the same layers over and over somehow implements different operations in each repetition. In this case, we can make an argument similar to the previous remark. Specifically, to fit a task that requires N parallel operations, we would need a model that has an information bandwidth of N/L, and applies a recurrence of N/L layers corresponding to N/L* serial *operations to compute the desired function. This should again be contrasted with pause-inference which requires only 1 set of parallel computations, and also a meagre information bandwidth of 1 token per intermediate vector.*

K Additional Related Work

Input-only tokens. The idea of using tokens that occur only as an input has found its use in various forms, most commonly as `<cls>` (Chang et al., 2023; Liu, 2019; Devlin et al., 2019), `<sep>` or `<mask>` in BERT (Devlin et al., 2019) and in a line of work on adding memory to transformers (Burtsev et al., 2020; Bulatov et al., 2022; Darcet et al., 2023).

Chain-of-thought (CoT) prompting and role of intermediate tokens. CoT prompting has been shown to significantly improve the reasoning capabilities of large language models (Wei et al., 2022; Nye et al., 2021; Lanchantin et al., 2023; Suzgun et al., 2023; Zelikman et al., 2022; Zhou et al., 2023; Wang et al., 2023b; Yao et al., 2023). Consequently, there has been a surge of interest in understanding the source of these CoT prompting gains. Feng et al. (2023); Merrill & Sabharwal (2023) theoretically argue that CoT aids by increasing the computational expressivity of the Transformer. Other empirical works (Turpin et al., 2023; Wang et al., 2023a; Madaan & Yazdanbakhsh, 2022) have shown that the generated intermediate reasoning steps can be unfaithful, not representing the model's true reasoning process. Wang et al. (2023a) empirically show that even incorrect reasoning steps can preserve $80\% - 90\%$ of the performance gains. However, Lanham et al. (2023) find that simply replacing the reasoning steps with filler tokens is unhelpful. As we argue, the model needs to be primed to process such tokens to help its computation.